Smart Fan Monitor

IoT-based temperature monitoring and fan control using ESP32 and Blynk

SUBMITTED BY	Sammeti Venkata Pranay · 2024002593
TEAM MEMBERS	B. Mokshith Reddy · 2024005604 R. Bala Aditya Kumar · 2024185482
COURSE	24EECE2031 · Introduction to IoT and Its Applications
DEPARTMENT	Electronics and Communication Engineering
INSTITUTION	GITAM (Deemed to be University), Visakhapatnam
YEAR	III Year · 2026–27

Contents

01	Abstract	03
02	Introduction	03
03	Objectives	03
04	Components used	04
05	Circuit and pin connections	04
06	Blynk IoT configuration	05
07	Working of the system	06
08	Software and code explanation	07
09	Problems faced and solutions	08
10	Results	08
11	Applications, limitations and future scope	09
12	Conclusion	10
13	References	10
14	Appendix: full source code	11

01 Abstract

The Smart Fan Monitor measures room temperature and humidity with a DHT11 sensor and raises an alarm when the temperature goes above 30°C.

An ESP32 DevKit V1 reads the sensor every 2 seconds, shows the temperature state on red and green LEDs, and sends live readings to the Blynk IoT mobile app over Wi-Fi.

When the room is above 30°C, a buzzer beeps and Blynk sends a heat_alert notification to the phone. The user turns the fan on from the Blynk app, and the buzzer stops immediately. A blue LED represents the fan in this prototype.

The system keeps working locally even when Wi-Fi or the Blynk server is unavailable, and it reconnects automatically.

02 Introduction

In hot climates such as coastal Andhra Pradesh, rooms often heat up during the day without anyone noticing until it becomes uncomfortable. A simple monitor that warns the user and lets them switch on a fan from their phone makes this easier to manage.

The Internet of Things (IoT) connects small devices such as sensors and microcontrollers to the internet so they can be monitored and controlled remotely. This project uses the ESP32, a low-cost microcontroller with built-in Wi-Fi, together with Blynk IoT, a cloud platform that provides a ready-made mobile dashboard.

The project combines sensing (DHT11), local indication (LEDs and buzzer), cloud communication (Blynk) and remote control (a switch in the app) in one small system.

03 Objectives

1. Measure room temperature and humidity using a DHT11 sensor.
2. Show the temperature state locally: red LED above 30°C, green LED at or below 30°C.
3. Sound an intermittent buzzer alarm when the temperature rises above 30°C.
4. Send temperature, humidity, fan status and a status message to the Blynk IoT app in real time.
5. Notify the user's phone through a Blynk event (heat_alert) when the temperature crosses 30°C.
6. Let the user switch the fan on or off from the Blynk app, with the buzzer stopping as soon as the fan is turned on.
7. Keep the device working and reconnecting automatically if Wi-Fi or Blynk is lost.

04 Components used

COMPONENT	QTY	PURPOSE
ESP32 DevKit V1 (30-pin)	1	Main controller with built-in Wi-Fi; reads the sensor, drives outputs, talks to Blynk
DHT11 sensor	1	Measures temperature (0–50°C, ±2°C) and humidity (20–90%, ±5%)
Active buzzer module	1	Audible heat alarm
Red LED	1	Indicates temperature above 30°C
Green LED	1	Indicates temperature at or below 30°C
Blue LED	1	Represents the fan (fan indicator)
220 Ω resistors	3	Current limiting for the LEDs
Breadboard and jumper wires	1 set	Circuit connections
Android phone (2.4 GHz hotspot)	1	Wi-Fi network and Blynk IoT app
USB cable	1	Power and programming

Software: Arduino IDE, ESP32 Arduino Core 3.x, Blynk library 1.3.x, Adafruit DHT sensor library, Blynk IoT mobile app and Blynk Console.

05 Circuit and pin connections

All components connect to the ESP32 on a breadboard and share a common ground.

COMPONENT	COMPONENT PIN	ESP32 PIN
DHT11	DATA	GPIO 4
DHT11	VCC	3.3 V
DHT11	GND	GND
Active buzzer	Signal / +	GPIO 25
Active buzzer	–	GND
Red LED	Anode (via 220 Ω)	GPIO 21
Green LED	Anode (via 220 Ω)	GPIO 18
Blue LED (fan)	Anode (via 220 Ω)	GPIO 26
All LEDs	Cathode	GND

The 220 Ω resistors limit LED current to about (3.3 V − 2 V) / 220 Ω ≈ 6 mA, well within the ESP32 GPIO limit.

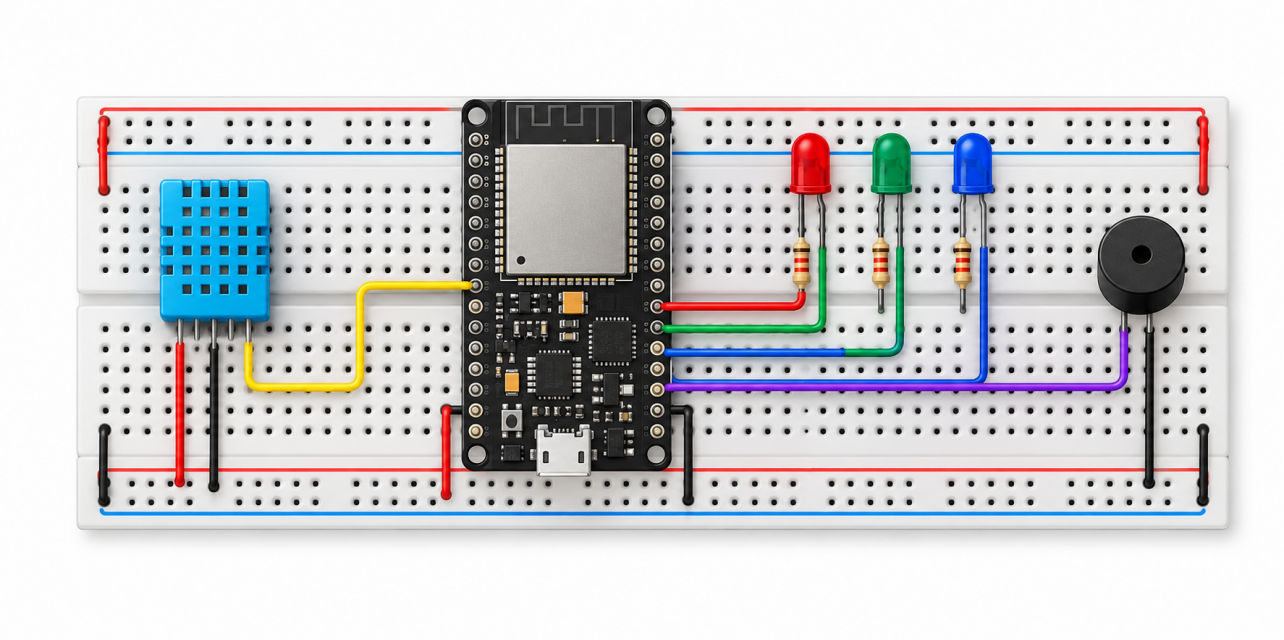


FIG. 1 · HARDWARE SETUP ON BREADBOARD (ILLUSTRATIVE)

06 Blynk IoT configuration

A Blynk template named Smart Fan Monitor holds six virtual datastreams and one event.

PIN	NAME	TYPE	RANGE / UNIT	DIRECTION	WIDGET
V0	Temperature	Double	°C	ESP32 → app	Gauge
V1	Humidity	Double	%	ESP32 → app	Gauge
V3	Fan Control	Integer	0–1	App → ESP32	Switch
V4	Buzzer Control	Integer	0–1	App → ESP32	Switch (buzzer test)
V5	Fan Speed	Integer	0–100 %	ESP32 → app	Value display
V7	Status	String	text	ESP32 → app	Label

Event: heat_alert, sent once each time the temperature rises above 30°C. It appears as a push notification on the phone.

The ESP32 connects using the Template ID, Template Name and Auth Token from the Blynk Console. These credentials are kept private and are not shown in this report.

07 Working of the system

The buzzer sounds whenever the room is above 30°C and the fan is off; turning the fan on from the app silences it immediately.

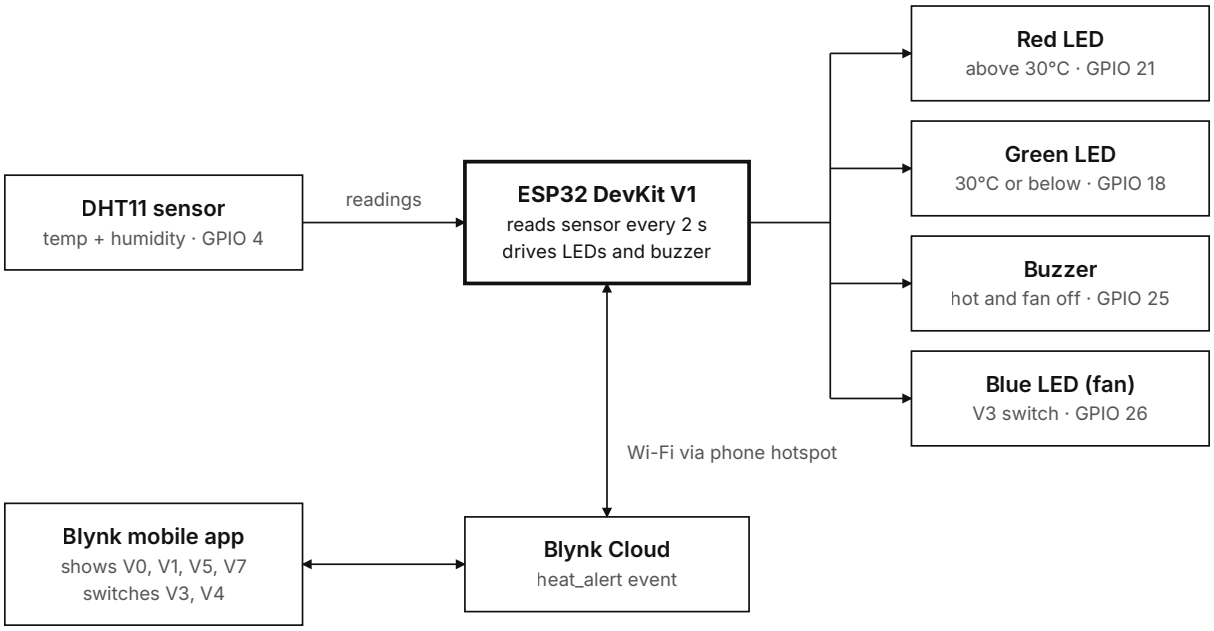


FIG. 2 · SYSTEM BLOCK DIAGRAM

The ESP32 sits at the centre: it reads the DHT11, drives the four outputs, and exchanges data with the Blynk app through the Blynk Cloud.

CONDITION	RED	GREEN	BUZZER	BLUE (FAN)	V5	V7 STATUS
Above 30°C, fan OFF	**ON**	OFF	**Beeping**	OFF	0%	ALERT! turn on fan
Above 30°C, fan ON	**ON**	OFF	OFF	**ON**	100%	fan ON, buzzer OFF
≤ 30°C, fan OFF	OFF	**ON**	OFF	OFF	0%	normal, fan OFF
≤ 30°C, fan ON	OFF	**ON**	OFF	**ON**	100%	normal, fan ON
Sensor error	OFF	OFF	OFF	as V3	as V3	DHT11 error

STEP BY STEP

1. On power-up the ESP32 sets all outputs OFF, connects to the phone hotspot (up to 20 s) and then to Blynk (up to 10 s).
2. On connecting, it reads the current V3 and V4 switch positions from the Blynk server, so the device always matches the app.
3. Every 2 seconds it reads the DHT11. Temperature and humidity go to V0 and V1.

4. Above 30°C the red LED turns on; at or below 30°C the green LED turns on.
5. Above 30°C with the fan off, the buzzer beeps (250 ms on, 250 ms off) and heat_alert is sent once.
6. When the user turns V3 ON, the blue LED turns on, V5 shows 100% and the buzzer stops at once.
7. If the fan is turned off while still above 30°C, the buzzer starts again as a reminder.
8. The fan stays in whatever state the user chose; temperature never switches it automatically.

V4 (buzzer test): turning V4 ON makes the buzzer beep for testing. It is ignored while the fan is ON, and turning the fan ON switches V4 back OFF in the app.

---

08 Software and code explanation

The sketch is written in Arduino C++ and uses a BlynkTimer to run tasks at fixed intervals instead of long delay() calls, so the buzzer, sensor and connection never block each other.

FUNCTION	RUNS	WHAT IT DOES
`setup()`	Once at power-up	Sets pin modes, attaches LED PWM (ledcAttach), starts DHT11, connects Wi-Fi and Blynk with time limits, starts timers
`readSensor()`	Every 2 s	Reads DHT11; ignores up to 4 failed reads in a row; calls applyOutputs() and sendToBlynk()
`applyOutputs()`	After every reading or switch change	Recalculates all outputs from scratch: red/green LEDs, fan LED, buzzer on/off, heat_alert
`updateBuzzer()`	Every 50 ms	Toggles the buzzer every 250 ms while the alarm is active
`sendToBlynk()`	After every change	Writes V0, V1, V5 and the V7 status text
`buildStatus()`	Inside sendToBlynk()	Builds the status message shown on V7
`connectBlynk()`	Every 15 s	Reconnects Wi-Fi or Blynk if either has dropped
`BLYNK_WRITE(V3)`	Fan switch changes	Sets the fan state; switches V4 OFF if needed
`BLYNK_WRITE(V4)`	Buzzer switch changes	Sets the buzzer test state
`BLYNK_CONNECTED()`	On every (re)connect	Syncs V3 and V4 from the server

KEY BUZZER RULE

```
buzzerOn = !fanOn && (hot || buzzerSwitch);
```

The buzzer can only sound while the fan is off, which is why turning the fan on silences it instantly.

DESIGN CHOICES

- Blynk.config() with Blynk.connect(timeout) is used instead of Blynk.begin(), which can block forever if the server is unreachable.

- Blynk.run() is called only while connected, so the local alarm keeps working offline.
- The ESP32 Core 3.x LEDC API (ledcAttach, ledcWrite) drives the blue LED, ready for PWM fan control later.

09 Problems faced and solutions

PROBLEM	CAUSE	SOLUTION
Device stuck at the Wi-Fi/ Blynk connection stage	Blynk.begin() blocks until it connects	Switched to Blynk.config() + Blynk.connect() with time limits and retries every 15 s
Buzzer kept beeping in the wrong state	Buzzer logic did not match the intended behaviour, and outputs kept old states	One applyOutputs() function now recalculates every output from scratch
Fan LED did not respond as expected	Brightness was tied to a scaled fan speed that was 0% below 27°C	Fan LED now follows the V3 switch directly (ON = full brightness)
Buzzer switch (V4) did nothing	No BLYNK_WRITE(V4) handler existed	Added the V4 handler
App switches and LEDs out of sync after reconnect	Device state was not refreshed from the server	Blynk.syncVirtual(V3, V4) runs on every connection
DHT11 read failures	Occasional NaN readings from the sensor	Readings every 2 s; up to 4 failures in a row are ignored before showing an error
Compilation error: WIFI_SSID not declared	Variable names did not match between two code versions	Kept the original ssid/pass names throughout

10 Results

The final system worked as designed in testing: the alarm, LEDs, app control and Blynk dashboard all behaved as described in the Working section.

- Temperature and humidity updated on the Blynk gauges every 2 seconds.
- Above 30°C, the red LED turned on and the buzzer beeped; at or below 30°C, only the green LED was on.
- Turning the fan switch (V3) ON in the app turned on the blue LED and stopped the buzzer immediately.
- The heat_alert notification reached the phone when the temperature crossed 30°C.

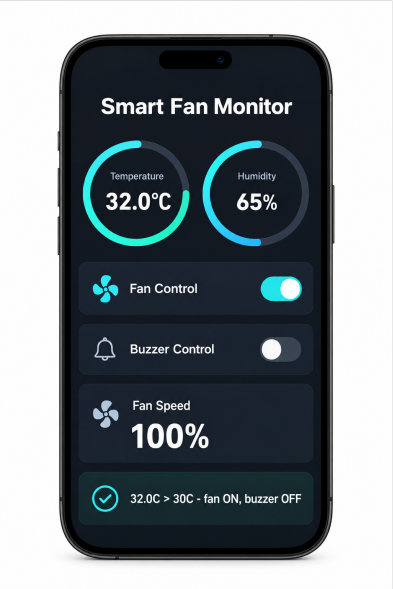


FIG. 3 · BLYNK DASHBOARD (ILLUSTRATIVE)

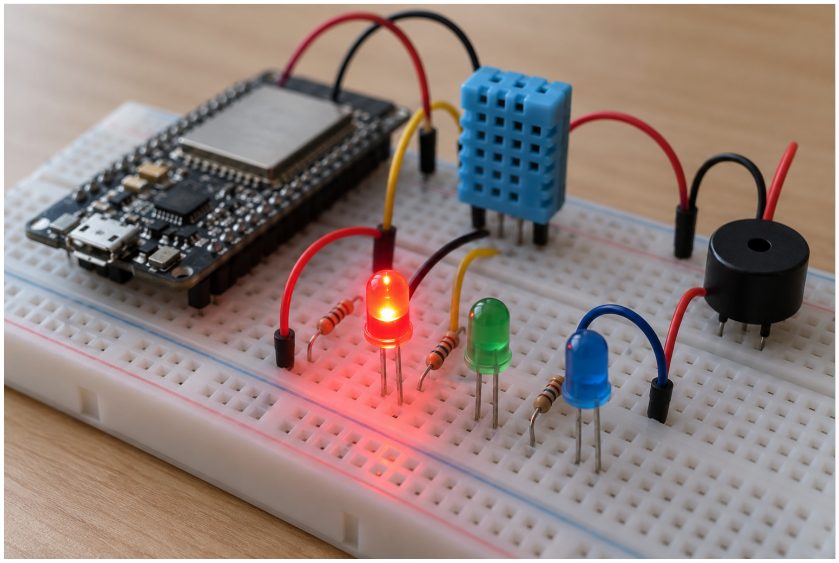


FIG. 4 · ALARM STATE: ABOVE 30°C, RED LED ON (ILLUSTRATIVE)

11 Applications, limitations and future scope

APPLICATIONS

- Room, hostel or classroom comfort monitoring
- Server rack or electronics cabinet heat alarm
- Greenhouse or storage room temperature warning
- A teaching example of ESP32 + cloud IoT control

LIMITATIONS

- The fan is represented by a blue LED; no real fan is driven yet.
- DHT11 accuracy is ±2°C with 1°C resolution, so readings near 30°C can flip between states.
- Remote control needs internet access through the phone hotspot.
- The threshold (30°C) is fixed in code and cannot be changed from the app.

FUTURE SCOPE

- Drive a real 5 V DC fan through a logic-level MOSFET module (e.g. D4184) on GPIO 27, with a flyback diode.
- Variable fan speed using PWM, shown on V5.
- Replace DHT11 with DHT22 or BME280 for better accuracy.
- Add hysteresis (e.g. alarm on above 30°C, off below 29°C) to stop flickering near the threshold.

- Add a threshold slider and a history chart in Blynk.

---

12 Conclusion

The Smart Fan Monitor successfully combines temperature sensing, a local alarm and remote control through the Blynk IoT app on a low-cost ESP32.

It warns the user when the room goes above 30°C, both on the device and through a phone notification, and lets them respond by switching on the fan from anywhere with internet access.

The project showed how important non-blocking code and a single, clear output-control function are in embedded IoT systems. With a MOSFET driver, the same design can control a real fan with only a small code change.

---

13 References

1. Espressif Systems, *ESP32 Series Datasheet*. espressif.com/en/support/documents/technical-documents
2. Espressif Systems, *Arduino Core for ESP32 (LEDC API)*. docs.espressif.com/projects/arduino-esp32
3. Blynk Inc., *Blynk IoT Documentation*. docs.blynk.io
4. Adafruit, *DHT Sensor Library for Arduino*. github.com/adafruit/DHT-sensor-library
5. Aosong Electronics, *DHT11 Humidity and Temperature Sensor Datasheet*.

14 Appendix: full source code

Credentials are replaced with placeholders.

```
#define BLYNK_TEMPLATE_ID   "YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "smart fan monitor"
#define BLYNK_AUTH_TOKEN    "YOUR_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

char ssid[] = "YOUR_HOTSPOT_NAME";
char pass[] = "YOUR_HOTSPOT_PASSWORD";

// Hardware pins
#define DHTPIN       4
#define DHTTYPE      DHT11
#define BUZZER_PIN   25
#define RED_LED      21
#define GREEN_LED    18
#define BLUE_LED     26     // fan indicator LED

const float TEMP_THRESHOLD = 30.0;   // > 30 = hot
const int   MAX_DHT_FAILS  = 5;

DHT dht(DHTPIN, DHTTYPE);
BlynkTimer timer;

bool fanSwitch    = false;   // V3
bool buzzerSwitch = false;   // V4 (test)

bool  sensorValid = false;
int   dhtFails    = 0;
float temperature = 0;
float humidity    = 0;

bool fanOn       = false;
int  fanSpeed    = 0;
bool buzzerOn    = false;
bool buzzerState = false;
bool alertActive = false;

unsigned long lastBeepMillis = 0;
const unsigned long BEEP_INTERVAL = 250;

void readSensor();
void applyOutputs();
void updateBuzzer();
void sendToBlynk();
void connectBlynk();
String buildStatus();

BLYNK_CONNECTED() {
  Serial.println("Blynk connected!");
  Blynk.syncVirtual(V3, V4);
  sendToBlynk();
}

BLYNK_WRITE(V3) {
  fanSwitch = (param.asInt() == 1);
  if (fanSwitch && buzzerSwitch) {
    buzzerSwitch = false;
    Blynk.virtualWrite(V4, 0);
  }
```

```
  applyOutputs();
  sendToBlynk();
}

BLYNK_WRITE(V4) {
  buzzerSwitch = (param.asInt() == 1);
  applyOutputs();
  sendToBlynk();
}

void applyOutputs() {
  bool hot = sensorValid && (temperature > TEMP_THRESHOLD);

  digitalWrite(RED_LED,   (sensorValid && hot)  ? HIGH : LOW);
  digitalWrite(GREEN_LED, (sensorValid && !hot) ? HIGH : LOW);

  fanOn    = fanSwitch;
  fanSpeed = fanOn ? 100 : 0;
  ledcWrite(BLUE_LED, fanOn ? 255 : 0);

  bool wasBuzzerOn = buzzerOn;
  buzzerOn = !fanOn && (hot || buzzerSwitch);

  if (buzzerOn && !wasBuzzerOn) {
    buzzerState = true;
    digitalWrite(BUZZER_PIN, HIGH);
    lastBeepMillis = millis();
  }
  if (!buzzerOn) {
    buzzerState = false;
    digitalWrite(BUZZER_PIN, LOW);
  }

  if (sensorValid) {
    if (hot && !alertActive) {
      alertActive = true;
      if (Blynk.connected()) {
        Blynk.logEvent("heat_alert", "Room temperature is above 30 degrees C! Turn on the fan.");
      }
    } else if (!hot) {
      alertActive = false;
    }
  }
}

void readSensor() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    dhtFails++;
    Serial.print("ERROR: DHT11 reading failed (");
    Serial.print(dhtFails);
    Serial.println(" in a row)");
    if (dhtFails >= MAX_DHT_FAILS) {
      sensorValid = false;
      applyOutputs();
      sendToBlynk();
    }
    return;
  }

  dhtFails    = 0;
  sensorValid = true;
  temperature = t;
  humidity    = h;

  applyOutputs();
  sendToBlynk();

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.print(" C | Humidity: ");
```

```
  Serial.print(humidity);
  Serial.print(" % | Fan: ");
  Serial.print(fanOn ? "ON" : "OFF");
  Serial.print(" | Buzzer: ");
  Serial.println(buzzerOn ? "ON" : "OFF");
}

void updateBuzzer() {
  if (buzzerOn) {
    if (millis() - lastBeepMillis >= BEEP_INTERVAL) {
      lastBeepMillis = millis();
      buzzerState = !buzzerState;
      digitalWrite(BUZZER_PIN, buzzerState ? HIGH : LOW);
    }
  } else {
    buzzerState = false;
    digitalWrite(BUZZER_PIN, LOW);
    lastBeepMillis = millis();
  }
}

String buildStatus() {
  if (!sensorValid) {
    return String("DHT11 error | fan ") + (fanOn ? "ON" : "OFF");
  }
  String s = String(temperature, 1) + "C ";
  if (temperature > TEMP_THRESHOLD) {
    s += fanOn ? "> 30C - fan ON, buzzer OFF" : "> 30C - ALERT! turn on fan";
  } else {
    s += "<= 30C - normal, fan ";
    s += fanOn ? "ON" : "OFF";
  }
  return s;
}

void sendToBlynk() {
  if (!Blynk.connected()) return;
  if (sensorValid) {
    Blynk.virtualWrite(V0, temperature);
    Blynk.virtualWrite(V1, humidity);
  }
  Blynk.virtualWrite(V5, fanSpeed);
  Blynk.virtualWrite(V7, buildStatus());
}

void connectBlynk() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Wi-Fi disconnected. Reconnecting...");
    WiFi.reconnect();
    return;
  }
  if (!Blynk.connected()) {
    Serial.println("Attempting Blynk connection...");
    if (Blynk.connect(5000)) {
      Serial.println("Blynk connection successful!");
    } else {
      Serial.println("Blynk connection failed; will retry.");
    }
  }
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, LOW);

  if (!ledcAttach(BLUE_LED, 5000, 8)) {
```

```
    Serial.println("ERROR: Blue LED PWM setup failed");
  }
  ledcWrite(BLUE_LED, 0);

  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, pass);
  Serial.print("Connecting to Wi-Fi");
  unsigned long startAttempt = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - startAttempt < 20000) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWi-Fi connected!");
  } else {
    Serial.println("\nWi-Fi connection timed out.");
  }

  Blynk.config(BLYNK_AUTH_TOKEN);
  if (WiFi.status() == WL_CONNECTED) {
    Blynk.connect(10000);
  }

  applyOutputs();

  timer.setInterval(2000L, readSensor);
  timer.setInterval(50L, updateBuzzer);
  timer.setInterval(15000L, connectBlynk);
}

void loop() {
  if (Blynk.connected()) {
    Blynk.run();
  }
  timer.run();
}
```